

Restaurant Management System Project Report

GitHub Repository Link: [Insert your GitHub repository URL here before final submission]

1. Introduction

1.1 Organisational Background

The chosen organisation for this project is a small independent restaurant that serves dine-in customers throughout the day. A business of this type usually has a simple operating structure, but the work still depends on accurate coordination between menu management, order taking, billing, and stock monitoring. In many small restaurants these tasks are handled with handwritten notes or very basic spreadsheets. That approach may work when customer traffic is low, but it becomes unreliable when the restaurant starts handling multiple tables at the same time. Staff can forget to update stock levels, write incomplete orders, or make arithmetic mistakes while preparing the final bill. These operational problems reduce service quality, create customer dissatisfaction, and make it harder for the owner to understand daily performance.

1.2 Problem Statement

The software created in this project addresses the practical problem of restaurant staff needing a quick and structured way to manage menu items and customer orders. The restaurant requires a program that can store menu information, record orders for different tables, reduce stock when dishes are sold, calculate bills consistently, and preserve completed transactions for later review. Without such a system, the business depends too heavily on memory and manual calculation. Errors in pricing, stock tracking, or order recording can affect both customer satisfaction and the financial control of the restaurant. The main goal is therefore to replace fragmented manual work with a compact, menu-driven application that is simple enough for a junior employee to use.

1.3 Purpose of the Program

The purpose of the Restaurant Management System is to support basic front-of-house operations in a clear and reliable way. The program allows a user to view the menu, add new menu items, restock existing items, create orders for tables, add items to those orders, generate a final bill, and review a sales summary. It is not intended to be a large enterprise platform. Instead, it is a small but realistic software solution that demonstrates how core Python programming concepts can be applied to a meaningful organisational problem. The design places emphasis on correctness, readability, and modularity so that the system can be understood and extended by other developers.

1.4 Scope

The scope of the project is deliberately focused on core restaurant operations. The application manages menu data, tracks stock, records active orders, calculates billing totals, and writes completed orders to storage files. It also stores bill output in a text archive and produces a simple sales report based on completed transactions. The project does not cover advanced features such as online reservations, employee payroll, kitchen display integration, supplier ordering, or a graphical user interface. These features are outside the current scope because the brief emphasises the use of foundational Python programming rather than full commercial deployment. Restricting the scope made it possible to produce a clean system that strongly demonstrates classes, objects, control structures, file handling, exception handling, and multi-file modular design.

1.5 Why This Topic Was Chosen

The restaurant topic suits an introductory programming assignment because it contains clear entities, repeatable workflows, and understandable business rules. Menu items, orders, bills, and stock records map naturally to classes and methods. The topic also creates realistic opportunities for validation, exception handling, and testing. For these reasons, it is both practical and academically appropriate.

Insert Screenshot Here: Figure 1. Main Menu Interface

2. System Design

2.1 Design Approach

The design of the program follows a modular and object-oriented structure. Instead of writing all logic in one file, the application is separated into domain models, storage logic, business rules, custom exceptions, and the user interface. This separation improves readability and reduces coupling between parts of the code. Domain classes represent the key real-world entities. Storage functions manage file input and output. Business logic connects menu items, orders, and bills. The console interface acts as the point where the user interacts with the system. This layered structure makes the program easier to maintain because each module has a clear responsibility.

2.2 Main Classes

The program includes more than the minimum required number of classes. `'MenuItem'` represents a food or drink item and stores the code, name, category, price, and stock. `'OrderLine'` represents a single ordered item and stores the code, item name, unit price, and quantity. `'CustomerOrder'` groups multiple order lines under one order ID and also stores table number, server name, status, and time created. `'Bill'` calculates tax, service charge, and grand total for a completed order. `'FileStorage'` manages all file input and output, while `'RestaurantSystem'` coordinates the whole workflow by loading data, creating orders, updating stock, checking out orders, and producing reports.

2.3 Relationships Between Classes

The classes are related in a way that reflects the domain. A `'RestaurantSystem'` contains many `'MenuItem'` objects in its menu collection. It also manages active and completed `'CustomerOrder'` objects. Each `'CustomerOrder'` contains a list of `'OrderLine'` objects. A `'Bill'` is generated from one completed `'CustomerOrder'`. The `'FileStorage'` class is used by `'RestaurantSystem'` to load and save data. The custom exception classes support validation and error reporting across the whole application.

2.4 UML-Style Class Summary

The following text diagram summarises the design:

`RestaurantSystem`

`- menu\_items: Dict[str, MenuItem]`

`- active\_orders: Dict[str, CustomerOrder]`

`- completed\_orders: List[CustomerOrder]`

`+ load\_state()`

`+ create\_order()`

`+ add\_item\_to\_order()`

`+ checkout\_order()`

`+ generate\_sales\_report()`

`MenuItem`

`- code, name, category, price, stock`

`+ update\_price()`

`+ has\_stock()`

`+ reserve\_stock()`

`+ restock()`

`CustomerOrder`

`- order\_id, table\_number, server\_name, status, lines`

`+ add\_line()`

`+ remove\_line()`

`+ calculate\_subtotal()`

```
`+ mark_paid()`
```

```
`OrderLine`
```

```
`- item_code, item_name, unit_price, quantity`
```

```
`+ increase_quantity()`
```

```
`+ decrease_quantity()`
```

```
`+ set_quantity()`
```

```
`+ line_total()`
```

```
`Bill`
```

```
`- order, tax_rate, service_rate`
```

```
`+ calculate_tax()`
```

```
`+ calculate_service_charge()`
```

```
`+ calculate_total()`
```

```
`+ to_text()`
```

2.5 Data Management and File Handling

Persistent storage is handled using flat files so that the solution remains simple and suitable for the module. The file `menu.csv` stores menu item records with fields for code, name, category, price, and stock. The file `orders.csv` stores completed order records, including serialized order lines. The file `bills.txt` stores readable bill output. When the program starts, it checks whether these files exist and creates them if necessary. Menu changes are written back to the CSV file, while completed orders and bill text are appended when checkout is completed. This design shows both CSV and text file handling without adding database complexity.

2.6 User Interface Design

The user interface is menu-driven and text-based. This is appropriate for the module because it keeps the focus on programming logic rather than graphics. The main menu displays numbered options for viewing the menu, adding items, creating orders, checking out, and printing a sales report. Clear prompts and concise output make the system easy to demonstrate and test.

Insert Screenshot Here: Figure 2. Order Creation and Item Entry Workflow

3. Implementation Overview

3.1 Use of Core Python Concepts

The implementation was built to reflect the programming concepts required by the brief. Object-oriented programming is central to the design. Classes and objects model menu items, order lines, customer orders, bills, and file storage. Each class contains meaningful attributes and methods so that data and behavior remain together. Small helper functions are also used in the console interface to reduce repetition and improve clarity. Dictionaries are used for fast lookup of menu items and active orders, while lists store order lines.

3.2 Control Structures

Control structures are used throughout the application. The main menu runs inside a ``while True`` loop so that the user can perform actions until choosing to exit. Conditional statements such as ``if``, ``elif``, and ``else`` select the correct action based on the menu choice. Iteration is used when printing menu items, scanning order lines, saving records, and calculating sales totals. These structures make the program interactive and suitable for repeated use.

3.3 File Input and Output

File input and output are implemented through the `'FileStorage'` class using Python's built-in `'csv'` module and standard text file handling. Menu items are loaded from CSV into objects, updated data is written back after stock or item changes, completed orders are appended to a second CSV file, and final bill text is written to a text archive. This demonstrates both structured and readable file output.

3.4 Exception Handling

Exception handling is included at both the model and interface levels. Custom exceptions such as `'ValidationError'`, `'NotFoundError'`, and `'StockError'` are used to represent different categories of application problems. For example, an error is raised if a user enters a negative quantity, requests a menu item code that does not exist, or attempts to order more units than are currently in stock. The main application loop catches these exceptions and prints friendly error messages instead of allowing the program to crash. This is important because it improves reliability and creates a better user experience.

3.5 Modularity and Multi-File Structure

The project is organised into multiple Python files to satisfy the brief and to follow good practice. `'models.py'` contains the domain classes, `'storage.py'` handles file persistence, `'system.py'` contains business coordination logic, `'exceptions.py'` defines custom exceptions, and `'main.py'` provides the console interface. This structure improves readability and makes future changes easier because responsibilities are separated cleanly.

3.6 Readability and PEP 8 Compliance

Readability was considered during implementation. Clear names were chosen for variables, methods, and classes, and docstrings were added to explain purpose. Line spacing, method organisation, and naming style were kept close to PEP 8 guidelines. Repetitive logic was reduced with helper methods so that important rules are defined in one place.

3.7 Business Rules Applied in Code

Several business rules were enforced directly in the code. An item cannot be sold if the requested quantity exceeds stock. An order cannot be checked out if it contains no items. Table numbers and quantities must be greater than zero. Menu item codes must be unique. The bill adds tax and a service charge using fixed rates. These rules are implemented through methods and validations inside the domain classes and controller.

3.8 Sample Workflow

In a normal workflow, the user launches the application, reviews the menu, creates an order for a table, and adds one or more items. Each added item reduces available stock. When the customer is ready to pay, the user selects checkout. The program produces a bill showing subtotal, tax, service charge, and grand total, then stores the completed order and bill in the relevant files. This workflow demonstrates the interaction between all major components.

Insert Screenshot Here: Figure 3. Final Bill Output

4. Testing and Demonstration

4.1 Testing Strategy

Testing was carried out using both automated and manual methods. Automated tests were written with `pytest` to validate core behavior in an isolated environment. These tests confirm that default menu items are seeded correctly, that orders can be created and checked out successfully, and that the sales report aggregates completed order data as expected. Manual testing was also important because the application is interactive. The console interface was tested with realistic restaurant tasks such as creating orders, adding items, checking out, and reviewing output. This combination provided broader confidence in correctness.

4.2 Example Functional Test Case

One representative functional test case was the creation and checkout of an order. In this scenario the user created order `ORD100` for table `3`, added two `Margherita Pizza` items, and selected checkout. The expected subtotal was 21.98 dollars because the price of one pizza is 10.99 dollars. The expected tax at 7 percent was 1.54 dollars, and the expected service charge at 5 percent was 1.10 dollars. The expected grand total was therefore 24.62 dollars. The system produced those values correctly, which confirms that the billing calculations are accurate for this scenario.

This test case was useful because it touched several parts of the system in one run. It proved that the program could create an order object, look up a menu item, reserve stock, calculate line totals, apply tax and service charge, and move the order into the completed order archive. It also verified that the printed bill remained readable to a human user, which is important in a restaurant context where the final output must be understood quickly.

4.3 Example Inputs and Outputs

The program prompts the user with numbered options and then requests input according to the chosen task. When creating an order, the user enters an order ID, table number, and server name. When adding an item, the user supplies the order ID, menu code, and quantity. Output includes confirmation messages, updated subtotals, and a formatted final bill listing ordered items, line totals, subtotal, tax, service charge, and grand total. This makes the demonstration easy to follow.

4.4 Error Handling Demonstration

Error handling was tested deliberately because restaurant staff may make mistakes during data entry. If the user enters a menu item code that does not exist, the system raises a `NotFoundError` and displays a message showing that the code was not found. If the user enters a quantity of zero or a negative number, the program raises a `ValidationError` with a message explaining that the quantity must be greater than zero. If the user attempts to add more units than are available in stock, the program raises a `StockError` and shows the remaining stock. These checks prevent corrupt data from entering the system and help the user correct mistakes immediately.

From a software quality perspective, these tests are important because they show the difference between validation and failure. A robust system should not simply crash when a mistake occurs. Instead, it should detect the problem, preserve the valid state of the program, and communicate clearly what the user needs to correct. In this project, the exception handling inside the menu loop supports that goal by catching domain-specific problems and returning the user to the application safely.

4.5 File Verification

Persistence was tested by inspecting the output files after checkout. The ``orders.csv`` file was checked to confirm that a completed order record had been appended correctly. The ``bills.txt`` file was inspected to ensure that a readable bill had been archived. The ``menu.csv`` file was reviewed to verify that stock had been reduced after a sale. These checks confirmed that file handling behaved as expected between sessions.

4.6 Testing Limitations

Although the system was tested well for an introductory project, some limitations remain. The automated tests cover the main business flow but not every edge case, such as corrupted input files or complex order editing scenarios. The application also assumes a single local user and does not address concurrency or security concerns. These limits are acceptable because the assignment focuses on core programming principles rather than production deployment.

Another limitation is that testing was performed with a small amount of sample data rather than a very large operational dataset. For a real restaurant, it would be useful to test the behavior of the reporting logic across many completed orders and to measure how the file structure performs over time. Even so, the present test set is adequate for demonstrating the correctness of the main educational features required by the module.

Insert Screenshot Here: Figure 4. Sales Report Output

Insert Screenshot Here: Figure 5. Error Handling Example

5. Reflection

5.1 Development Experience

This project demonstrated how a realistic business scenario can be translated into a structured software solution using core Python concepts. One of the most useful lessons was that even a small problem becomes easier to manage when it is divided into classes with clear responsibilities. Separating models, storage, exceptions, and system coordination made the application easier to test and extend.

5.2 Challenges Encountered

A key challenge during development was choosing a storage method that remained simple but still useful. A full database would have been beyond the scope of the assignment, while plain unstructured text would have made reporting difficult. The chosen solution was to use CSV files for menu and completed order records and a text file for bill output. Another challenge was maintaining correct stock levels while orders were active. The program reduces stock as soon as an item is added, which prevents overselling but also shows that future order-editing features would need careful design.

5.3 Lessons Learned

The project reinforced several lessons. Validation should happen close to the data it protects, which is why quantity and price checks are placed inside model methods. Custom exceptions improve clarity because they distinguish between missing data, invalid input, and stock problems. File handling is also more reliable when centralised inside one storage class. The assignment further showed the value of automated tests, while also confirming that manual testing is still important for interactive programs.

An additional lesson was the value of designing around change. Even though the current version is small, choosing modular files and focused classes means the program can grow without becoming disorganised immediately. For example, a future developer could replace text-based input with a graphical interface while still reusing the existing business logic and storage layer. Thinking in this way encourages better long-term habits in software development.

5.4 Future Improvements

Several extensions could improve the application in future work. A graphical user interface could make the system easier for non-technical staff. A database such as SQLite could improve reliability and allow better querying of order history. The system could also support editing or cancelling active orders, user roles, supplier management, receipt printing, and stronger reporting such as best-selling items or low-stock alerts.

It would also be useful to add stronger validation around stored files. At the moment, the program assumes that existing CSV records follow the expected format. Future versions could add dedicated recovery features, better logging, and administrative tools for correcting damaged records. These additions would move the system further from a classroom prototype toward a dependable small-business utility.

5.5 Final Evaluation

Overall, the final system meets the main purpose defined at the start of the project. It provides a clear restaurant workflow, demonstrates the required Python concepts, and shows how software can improve accuracy in a small organisation. The program does not solve every operational problem of a restaurant, but it achieves its limited scope effectively and demonstrates how core programming ideas can work together in one coherent solution.

Conclusion

The Restaurant Management System is an effective response to the assignment brief. It models a realistic organisational problem, implements the required features in Python, and presents a

maintainable structure that could support future growth. The project shows that even a compact console program can deliver practical value when it is designed carefully and tested systematically.